

Relatório 3 - Prática de configuração do ambiente e do projeto

Kaíque Medeiros Lima

1. Descrição da atividade

a. Preparação das ferramentas

A atividade começa com a preparação do ambiente que vai ser usado no desenvolvimento do projeto. Os comandos `node -v` e `pnpm -v` mostram as versões instaladas, enquanto o `gh auth status` confirma a conexão da conta com o GitHub por SSH. Essa verificação é simples, mas ajuda a conferir se as ferramentas básicas já estão disponíveis antes de começar a instalar as dependências.

O Node.js permite executar JavaScript fora do navegador e dá suporte às ferramentas do projeto. Já o pnpm fica responsável por instalar e organizar os pacotes. O GitHub CLI entra na parte de integração com o repositório, permitindo consultar a autenticação pelo próprio terminal. Cada ferramenta acaba cuidando de uma parte diferente da preparação.

```
~/web-fastcamp-lamia/CARD 03 main*
base > node -v                                     (base)
v22.23.1

~/web-fastcamp-lamia/CARD 03 main*
base > pnpm -v                                     (base)
11.22.0

~/web-fastcamp-lamia/CARD 03 main*
base > gh auth status                             (base)
github.com
✓ Logged in to github.com account noirelab (/home/noirelab/.config/gh/hosts.yml)
- Active account: true
- Git operations protocol: ssh
- Token: gho_*****
- Token scopes: 'admin:public_key', 'gist', 'read:org', 'repo'
```

b. Criação do projeto com Next.js

O projeto foi iniciado pelo comando `pnpm create next-app@latest meu-projeto --typescript --tailwind --eslint --app --src-dir`. Com essas opções, a estrutura inicial já é criada com TypeScript, Tailwind CSS, ESLint, App Router e a pasta `src`. Isso evita ter que montar toda a configuração do zero e deixa uma base organizada para continuar os próximos cards.

A instalação registrada inclui o Next.js, o React e o React DOM, além dos pacotes usados durante o desenvolvimento. O TypeScript entra na definição dos tipos, o Tailwind na estilização e o ESLint na análise do código. A criação termina com a geração dos tipos das rotas e a mensagem de sucesso do projeto.

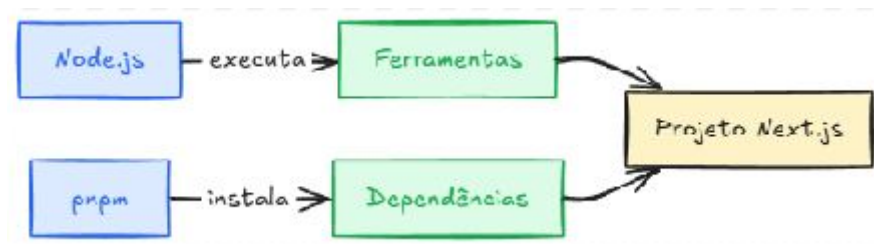


Figura 1 - Ferramentas na preparação do projeto. Autoria própria, Excalidraw.

c. Organização das branches

Na parte de versionamento, foi criada a branch develop com git checkout -b develop e feito o envio para o GitHub com git push -u origin develop. Assim, o desenvolvimento passa a ter uma branch própria, separada da main. Também foi registrada a criação de uma regra de proteção para a branch main nas configurações do repositório.

O README apresenta o fluxo em que a main fica como branch estável, a develop concentra o desenvolvimento e cada tarefa pode sair de uma branch feature. Essa divisão ajuda a organizar as mudanças e a revisar o que vai entrar na versão principal. A ideia é que o histórico acompanhe o trabalho de forma mais clara, principalmente quando várias alterações acontecem ao mesmo tempo.



Figura 2 - Fluxo de branches descrito no README. Autoria própria, Excalidraw.

d. Padronização do código e dos commits

Foram adicionados o Prettier e o plugin de Tailwind como dependências de desenvolvimento. Essas ferramentas ajudam a manter a formatação do código e a organização das classes de estilo. Isso se relaciona com as boas práticas do card anterior, porque um padrão compartilhado facilita a leitura e diminui diferenças que são só de formatação.

Também aparecem a instalação do Husky e do commitlint, a inicialização dos hooks e a criação da configuração baseada em Conventional Commits. O hook commit-msg chama o commitlint para verificar a mensagem do commit. Com esse padrão, prefixos como feat, fix e docs indicam melhor o tipo de alteração feita no repositório.

```
~/web-fastcamp-lamia/CARD 03 develop*
base > pnpm add -D prettier prettier-plugin-tailwindcss (base)
Packages: +2
++
Packages are cloned from the content-addressable store to the virtual store.
Content-addressable store is at: /home/noirelab/.local/share/pnpm/store/v1
Virtual store is at: node_modules/.pnpm
Progress: resolved 2, reused 0, downloaded 2, added 2, done

devDependencies:
+ prettier 3.9.6
+ prettier-plugin-tailwindcss 0.8.1

Done in 752ms using pnpm v11.22.0
```

e. TypeScript, variáveis de ambiente e documentação

O `tsconfig.json` registrado está com `strict` habilitado e define o atalho `@/*` para os arquivos dentro de `src`. O modo estrito deixa a checagem dos tipos mais exigente, enquanto o atalho simplifica os imports entre as pastas. A configuração também inclui arquivos TypeScript, TSX e os tipos gerados pelo Next.js.

O arquivo `.env.example` apresenta a variável `NEXT_PUBLIC_API_URL` com um endereço local para a API. Já o README reúne a stack, os comandos de instalação e execução, a organização das pastas e o fluxo de branches e commits. Essa documentação serve como um ponto de partida para quem precisar preparar o mesmo projeto depois, sem depender de lembrar cada comando usado na configuração.

```
~/web-fastcamp-lamia/CARD 03/meu-projeto main*
base > pnpm add -D @commitlint/cli @commitlint/config-conventional husky (base)

# Inicializar husky
pnpm husky init

# Criar hook de commit-msg
echo 'pnpm commitlint --edit "$1"' > .husky/commit-msg

# Criar commitlint.config.js
echo "module.exports = { extends: ['@commitlint/config-conventional'] };" > commitlint.config.js
✓ Lockfile passes supply-chain policies (verified 1m ago)
WARN deprecated eslint@9.39.5: This version is no longer supported. Please see https://eslint.org/version-support for other options.
Packages: +58
Progress: resolved 493, reused 370, downloaded 51, added 58, done

devDependencies:
+ @commitlint/cli 21.2.2
+ @commitlint/config-conventional 21.2.2
+ husky 9.1.7

Done in 2.5s using pnpm v11.22.0

~/web-fastcamp-lamia/CARD 03/meu-projeto main*
base > printcli 6.png (base)
pronto: 6.png 1920x137
```

2. Conclusões

A prática mostra que preparar um projeto envolve mais do que instalar um framework. As ferramentas, os tipos, a documentação e a organização do Git fazem parte da mesma base de trabalho. Com essa estrutura, fica mais fácil continuar o desenvolvimento e entender onde cada coisa deve ficar. A relação com o card anterior aparece justamente nesse cuidado de definir responsabilidades e padrões desde o começo, sem precisar deixar toda a organização para quando o projeto já estiver maior.

3. Referências

Material do card.